Missouri State University

Welcome to CSC 365

Internet Programming

Dr. Yifan Zhang

Manging State

Notice how the state disappears when flipping through the pages? Why is this?

How do we talk back to our parent? How do siblingstalk to each other?

- Using callbacks
- Using useContext
- Using sessionStorage and localStorage
- Using cookies and an API (next time!)
- Using third-party libraries like (on your own!)...Redux, Recoil, MobX, XState

Cookies vs. Session vs. Local

These exist in your browser! They will persist even after a page refresh.

- Wordle uses sessionStorage and localStorage to store game state.
- Facebook uses cookies to track you.
- Vanguard uses cookies to store temporary session/login credentials.

Cookies vs. Session vs. Local

Criteria	Local Storage	Session Storage	Cookies
Storage Capacity	5-10 mb	5-10 mb	4 kb
Auto Expiry	No	Yes	Yes
Server Side Accessibility	No	No	Yes
Data Transfer HTTP Request	No	No	Yes
Data Persistence	Till manually deleted	Till browser tab is closed	As per expiry TTL set

Cookies vs. Session vs. Local

Type	Notes
Cookies	Can be set programmatically, but typically set by server through a `Set-Cookie` header
Session	Set programmatically via `sessionStorage`, typically used with form data.
Local	Set programmatically via `localStorage`, typically used with long-lasting data.

Cookies vs. Session vs. Local

These are all just key-value pairs of *strings*! Can use `setItem` and `getItem` on both!

Type	Example
Session	`sessionStorage.setItem('name', 'Cole')`
Local	`localStorage.getItem('lastLogin')`

sessionStorage and localStorage

Reminder: sessionStorage and localStorage onlystore *strings*! Use JSON.parse and JSON.stringify!

INCORRECT Way

```
sessionStorage.setItem('nums', [2, 6, 19])
const storedNums = sessionStorage.getItem('nums')
```

Correct Way

```
sessionStorage.setItem('nums', JSON.stringify([2, 6, 19]))
const storedNums = JSON.parse(sessionStorage.getItem('nums'))
```

sessionStorage and localStorage

Remember, sessionStorage and localStorage are *browser mechanisms* that exist *outside of React*.

They do not trigger re-renders.

Do you need to re-render? You will also need tochange a state variable.

sessionStorage and localStorage tiny example

```
function Notes() {
  const [text, setText] = React.useState('');

  // 1) Load from storage on mount
  React.useEffect(() => {
    const saved = localStorage.getItem('notes') ?? '';
    setText(saved);
  }, []);

  // 2) Save to storage and update state so UI re-renders
  const handleChange = (e) => {
    const value = e.target.value;
    setText(value);                 // triggers re-render
    localStorage.setItem('notes', value);
  };

  return <textarea value={text} onChange={handleChange} />;
}
```

Third Party Libraries

Each has its own unique way of managing state. Examples include...

- Redux
- Recoil
- MobX
- XState

Note! These come and go. See flux.

Client Vs. Server-Side Storage

These are all examples of doing client-side storage.
What if we want to persist data long-term?
Server-side storage with PUT, POST, and DELETE!

We'll explore this next time.

React Router

Types of Routers

- BrowserRouter: What you typically think of!
- MemoryRouter: Same as BrowserRouter, but thepath is hidden from the browser in memory!
- HashRouter: Support for deploying SPAs. Use this if you are deploying to GitHub Pages!
- StaticRouter: Used for server-side rendering.
- NativeRouter: We'll use react-navigation instead!

Install: npm install react-router-dom

Routing

Using a Router, Routes, and Route!

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Layout />}>
      <Route index element={<Home />} />                    // default child (/)
      <Route path="about-us" element={<AboutUs />} />       // /about-us
      <Route path="other-info" element={<OtherInfo />} />   // /other-info
      <Route path="*" element={<Home />} />                 // catch-all /404
    </Route>
  </Routes>
</BrowserRouter>
```

Browser outlet

<Outlet/> shows the component returned by the child route! e.g. in Layout we may see...

```
function Layout() {
  return (
    <>
      <Navbar bg="dark" variant="dark">
        {/* nav links */}
      </Navbar>

      <Outlet />  {/* child route renders here */}
    </>
  );
}
```

Navigable Components

Notice how each route maps to a component.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="about-us" element={<AboutUs />} />
  <Route path="other-info" element={<OtherInfo />} />
</Routes>
```

useNavigate Hook

Useful for programmatic navigation!

```
import { useNavigate } from 'react-router-dom';

export default function OtherInfo() {
  const navigate = useNavigate();        // 1) get a navigate function
  const handleClick = () => {
    navigate('/home');                   // 2) push “/home” onto history
  };

  return (
    <div>
      <h2>Other Info!</h2>
      <Button onClick={handleClick}>Back to Home</Button>
    </div>
  );
}
```

Navigation

Navigation for a BrowserRouter is done via URLs.

```
<Navbar bg="dark" variant="dark">
  <Nav className="me-auto">
    <Nav.Link as={Link} to="/">Home</Nav.Link>
    <Nav.Link as={Link} to="/about-us">About Us</Nav.Link>
    <Nav.Link as={Link} to="/other-info">Other Info</Nav.Link>
  </Nav>
</Navbar>

<Outlet />
```

ICA 12